

DAY 10 · FastAPI learning notes

FastAPI + SQLAlchemy: SQL Databases

Study notes on the SQL (Relational) Databases tutorial · quiz score 6/10 · 9 Oct 2026 · now using PostgreSQL

Learning objectives

After working through these notes you should be able to:

- ✓ Describe how FastAPI, SQLAlchemy and the database work together (engine, session, table models).
- ✓ Load database credentials from `.env` and build a safe connection URL with `URL.create`.
- ✓ Explain exactly what `session.exec`, `.first()`, `.all()` and `session.get` return.
- ✓ Write create / read / update / delete endpoints with `add`, `commit`, `refresh` and `delete`.
- ✓ Predict which errors become 422, 404 or 500, and why one model per job is safer.

Note

This PDF keeps every section of [FastAPI_SQLAlchemy_Study_Notes \(1\).pdf](#) (organized by topic; boxes marked **Your quiz mistake** are things you got wrong on the quiz, and every section ends with a line to remember). New sections were added for the `.env` configuration and a CRUD walkthrough, and “Fix your file” was updated to the current state of [day10/main.py](#).

1. The big picture

Your Python code never writes SQL directly. SQLAlchemy sits in the middle and translates both ways: classes and objects become SQL, and the rows the database returns become Python objects again.

```
Client  --JSON--> FastAPI  --objects--> SQLAlchemy  --SQL--> Database
        <-JSON---          <-objects---          <-rows---
```

SQLAlchemy = **SQLAlchemy** (talks to the database) + **Pydantic** (validation and JSON). A class with `table=True` is a real table. Without it, it's just a data shape. Add `echo=True` to `create_engine` to print every SQL statement it runs.

	Engine	Session
What	holds the connections (connection pool)	one unit of work
How many	one per app	one per request, opened and closed by <code>get_session()</code>
Lifetime	created when Python imports your file, destroyed when the process ends (Ctrl+C). In dev mode every save reloads the app and makes a new one.	one request; the code after <code>yield</code> runs after the response is sent

Your quiz mistake

You said the engine stops when there's an error. A failed request only affects its own session.

Remember

Engine lives as long as the app. Session lives as long as one request.

2. Configuration: credentials from .env (new)

Your app connects to **PostgreSQL** (database `heroes_db` on `localhost:5432`) with the `psycopg3` driver. The username and password are secrets, so they live in a `.env` file in the project root – which is listed in `.gitignore` and never committed. `.env.example` shows the expected keys with placeholder values.

```
# .env (project root, NOT committed)
DB_USERNAME=your_database_username
DB_PASSWORD=your_database_password
```

day10/main.py

```
load_dotenv()  # copies KEY=value lines from .env into the environment

password = os.getenv("DB_PASSWORD")
username = os.getenv("DB_USERNAME")

if not username or not password:
    raise RuntimeError("DB_USERNAME and DB_PASSWORD must be set. ...")

database_url = URL.create(
    drivername="postgresql+psycopg",
    username=username,
    password=password,
    host=os.getenv("DB_HOST", "localhost"),
    port=int(os.getenv("DB_PORT", "5432")),
    database=os.getenv("DB_NAME", "heroes_db"),
)
engine = create_engine(database_url)
```

- `os.getenv` returns `None` when a variable is missing, so the app now stops early with a clear message instead of failing later with a confusing login error.
- **Why `URL.create`?** An f-string such as `f"postgresql+psycopg://{username}:{password}@localhost..."` breaks when the password contains `@`, `:`, `/` or `#`, because those characters mean something in a URL. `URL.create` keeps each part separate and escapes it. (Verified with a test password containing `@`, `:`, `/` and `#`.)
- The variable was renamed from `sqlite_url` to `database_url`, because it is a PostgreSQL URL. The unused `connect_args = {"check_same_thread": False}` (a SQLite-only option left over from the tutorial) was removed.
- `DB_HOST`, `DB_PORT` and `DB_NAME` are optional; their defaults are exactly the values you used before.

Remember

Secrets go in `.env` (ignored by git); `.env.example` (committed) only shows the keys.

3. Running queries: what you actually get back

`session.exec(query)` does **not** return a `Hero`. It returns a result object (`ScalarResult`): an unopened box of rows. You then choose how to open it.

Why two steps? To SQL, a query can match 0, 1 or many rows – even `WHERE id = 5`. Only you know what you expect, so you say it: one row, the first row, or all rows. The box also lets you stream rows without loading everything into memory.

Code	Type you get	If nothing matches
<code>session.exec(select(Hero))</code>	<code>ScalarResult[Hero]</code> (the box)	empty box, still truthy

Code	Type you get	If nothing matches
<code>... .first()</code>	Hero None	None
<code>... .all()</code>	list[Hero]	[]
<code>... .one()</code>	Hero	raises an error (also if more than one)
<code>session.get(Hero, 5)</code>	Hero None	None

get vs select. `session.get` only looks up by primary key and is the shortcut for that. It may even skip the database if the session already has that object. Use `select(...).where(...)` for anything else: filter by name, age range, sorting, pagination. Use `.first()` when you expect at most one row, `.all()` for a list.

Your quiz mistake

You said `exec` returned a Hero and the 404 would fire. The box is truthy even when empty, so `if not hero` never fires; FastAPI then fails to turn the box into a Hero → **500, not 404**.

Remember

`exec` gives a box → open it with `.first()` or `.all()`. `get(Model, id)` gives the object.

4. Checking whether a record exists

Condition	True when hero is...
<code>if not hero:</code>	anything falsy: <code>None</code> , <code>0</code> , <code>""</code> , <code>[]</code> , <code>{}</code>
<code>if hero is None:</code>	exactly <code>None</code> , nothing else
<code>if hero is not None:</code>	anything except <code>None</code> , including <code>0</code> and <code>""</code>

For a Hero object both `not hero` and `hero is None` work, because model objects are always truthy. Still prefer `is None`: it says exactly what you mean, and it stays correct when a query returns a plain value like a count of `0` or an empty string, which `not` would wrongly treat as “missing”.

```
hero = session.get(Hero, hero_id)
if hero is None:           # not found
    raise HTTPException(status_code=404, detail="Hero not found")
```

Remember

Checking for ‘not found’? Write `is None`.

5. When things go wrong: errors become 500

This is your biggest gap. **Pydantic checks the shape** of the request: types and required fields. It never looks inside the database. **The database checks the rules**: unique keys, NOT NULL, existing columns. If the database rejects something and you don't catch it, FastAPI returns **500 Internal Server Error**.

What happened	Who catches it	Client gets
wrong type or missing field	Pydantic	422
you raised <code>HTTPException(404)</code>	your code	404
duplicate primary key or unique value	database → <code>IntegrityError</code>	500
<code>None</code> into a NOT NULL column	database → <code>IntegrityError</code>	500

What happened	Who catches it	Client gets
column doesn't exist in the table	database → <code>OperationalError</code> / <code>ProgrammingError</code>	500

Your quiz mistake

For a POST with an existing id you guessed 424. Validation passes (it's a valid int), the DB rejects the duplicate → `IntegrityError` → 500. 424 means “Failed Dependency” and is unrelated.

How to handle it. For `id`: don't accept it from the client at all – that's why `HeroCreate` has no `id`. For a field the client must send but that has to be unique (like an email): catch `IntegrityError` around the commit, call `session.rollback()`, and return **409 Conflict**. Don't check first with `get`: two requests can both check, both see “free”, and both insert.

Remember

Pydantic checks shape, the DB checks rules, and any uncaught exception is a 500.

6. Multiple models: one model per job

Version 1 used one class for everything, so the client could send `id` and `secret_name` leaked in every response. Version 2 splits it:

Model	Job	id	secret_name
<code>HeroBase</code>	shared fields (parent)	–	–
<code>Hero</code>	the table (<code>table=True</code>)	<code>int None</code>	yes
<code>HeroCreate</code>	POST body	no	yes (accepted)
<code>HeroUpdate</code>	PATCH body, all optional	no	optional
<code>HeroPublic</code>	the response	<code>int</code>	no (never returned)

`Hero.model_validate(hero_create)` copies the input into a table object.

`response_model=HeroPublic` filters the output. It's used instead of `-> HeroPublic` because the function really returns a `Hero`, and the type checker would complain.

Why two different id types? Same field, different moments. A `Hero` exists before saving, when it has no id yet, so it must allow `None`. `HeroPublic` is only used after saving, when the id always exists, so `id: int` is a promise to clients that they never need to check for `None`.

Your quiz mistake

You explained only the `Hero` half. The point of `HeroPublic`'s type is the **guarantee**.

Passwords follow the `secret_name` pattern: accept them, never return them, and hash them before storing.

Remember

Before saving, id may be `None`. After saving, it's guaranteed, so promise it.

7. CRUD walkthrough of your endpoints (new)

Create

```
@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate, session: SessionDep):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero
```

Step	What happens
<code>Hero.model_validate(hero)</code>	copy the validated input into a table object (<code>id</code> is still <code>None</code>)
<code>session.add(db_hero)</code>	stage it in the session – nothing is saved yet
<code>session.commit()</code>	send the INSERT and make it permanent
<code>session.refresh(db_hero)</code>	reload from the database, so <code>db_hero.id</code> is now filled in
<code>return db_hero</code>	FastAPI filters it through <code>HeroPublic</code> → <code>secret_name</code> is dropped

Endpoint	Key line	Not found
GET /heroes/	<code>session.exec(select(Hero).offset(offset).limit(limit)).all()</code>	returns []
GET /heroes/{hero_id}	<code>session.get(Hero, hero_id)</code>	404
PATCH /heroes/{hero_id}	<code>hero.model_dump(exclude_unset=True)</code> → <code>hero_db.sqlmodel_update(...)</code> → add/commit/refresh	404
DELETE /heroes/{hero_id}	<code>session.delete(hero)</code> → <code>session.commit()</code>	404

Verified against a real PostgreSQL server: POST → `{"name": "Eyad", "age": 21, "id": 1}` (no `secret_name`), PATCH `{"name": "Ali"}` changes only the name, DELETE → `{"ok": true}`, then GET → 404.

`limit: Annotated[int, Query(le=100)] = 100` caps a page at 100 rows – the Day 2 validation rules in action. `get_session` + `SessionDep` is the Day 7 `yield` pattern combined with the Day 6 reusable `Annotated` alias.

8. PATCH and `exclude_unset`

`hero.model_dump()` returns every field, filling unset ones with their defaults (`None`).

`hero.model_dump(exclude_unset=True)` returns only the fields the client actually sent. That's how you keep untouched fields from being overwritten. Example database row: `name=Eyad, age=21, secret_name=esa`.

PATCH body	With <code>exclude_unset</code>	Without <code>exclude_unset</code>
<code>{}</code>	nothing changes	tries to set everything to <code>None</code> → 500
<code>{"age": null}</code>	age becomes NULL (client asked for it)	also nulls name and <code>secret_name</code> → 500
<code>{"name": "Ali"}</code>	only name changes	<code>secret_name=None</code> breaks NOT NULL → 500, nothing saved

Your quiz mistake

You said age and `secret_name` both become `None`. The dump does say that, but `secret_name` is NOT NULL in the database, so the commit fails, the transaction is rolled back, and age stays 21.

Remember

`exclude_unset` separates “the client didn't mention it” from “the client sent null”.

9. Changing the schema after the table exists

Editing the class only changes Python. The table in the database is a separate thing. `create_all` only creates tables that are **missing**. If `hero` exists, it skips it completely and never adds, removes or alters columns.

So after adding `email: str` and restarting: no new column. The next POST runs an INSERT that includes `email` → the database reports the column does not exist → 500.

Your quiz mistake

You got the first part, but missed the error on the next POST. And deleting the database (or the SQLite file you used at first) is only OK in development – in production it wipes every row.

Doing it by hand from Python (fine for learning):

```
from sqlalchemy import text
with engine.begin() as conn:          # begin() commits automatically at the end
    conn.execute(text("ALTER TABLE hero ADD COLUMN email TEXT"))
```

Existing rows have no email, so adding a NOT NULL column needs a DEFAULT value, or the database refuses.

sqlite3 vs the engine. Python's built-in `sqlite3` module only talks to SQLite. The SQLAlchemy engine is database-agnostic: the same code works on PostgreSQL by changing the URL (exactly what you did when you moved from SQLite to PostgreSQL), and it manages connections and transactions for you.

Optional / later

Why migrations (Alembic) in production instead of manual ALTER TABLE? Each change becomes a versioned script stored in git. It runs the same way on your laptop, your teammates' machines and the server, can be undone with a downgrade, and Alembic can generate it by comparing your models to the database. Manual commands are easy to forget, run twice, or run differently on each machine.

Remember

`create_all` is a dev convenience. Production schema changes go through migrations.

10. Defining the same table twice

```
InvalidRequestError: Table 'hero' is already defined for this MetaData instance.
Specify 'extend_existing=True' to redefine options and columns.
```

Every class with `table=True` registers its table in `SQLModel.metadata`. A file that defines `Hero` twice (v1 and v2) crashes on import, and the app never starts. `extend_existing=True` only silences the error; the real problem is two definitions of one table. Routes have a similar issue: with the same method and path registered twice, the first one wins and the second is dead code.

Your quiz mistake

You said “the first one is used”. That's true for routes, but the table error stops everything before any route runs.

Remember

One table class per table. Keep versions in separate files (v1.py, v2.py) or commented out.

11. Fix your file: status in the current day10/main.py

Problem from the quiz review	Status now
<code>exec</code> without <code>.first()</code> → 404 never fires	Fixed - v2 <code>read_hero</code> uses <code>session.get</code> ; the v1 version is commented out
comment says "if hero is not None" (backwards)	only in the commented-out v1 <code>delete_hero</code> ; explained in <code>main_commented.py</code>
<code>Hero</code> defined twice → crash on import	Fixed - v1 <code>Hero</code> is commented out
pagination without ORDER BY: page order not guaranteed	open - add <code>.order_by(Hero.id)</code>
<code>@app.on_event</code> is the older style	open - fine for now; <code>lifespan</code> is current
hard-coded/insecure connection string	Fixed - <code>.env</code> + <code>URL.create</code> , clear error if missing

12. Practice: break it on purpose

Run with `echo=True`, use `/docs`, and read the traceback each time:

- 1 POST a hero with an id that already exists → `IntegrityError`.
- 2 Remove `exclude_unset`, then PATCH only the name → NOT NULL failure.
- 3 Add `email: str` to Hero without changing the table, then POST → missing-column error.
- 4 Recreate the original v1 `read_hero` (without `.first()`) with a missing id → 500. Then fix it and get 404.
- 5 Stretch: add a unique email and return 409 on duplicates.

Review in this order: 1) what `exec` returns · 2) result types · 3) get vs select · 4) None, truthiness and 404 · 5) errors → 500 · 6) `exclude_unset` · 7) schema changes and migrations · 8) duplicate tables.

13. Revision summary

- SQLAlchemy = SQLAlchemy + Pydantic; `table=True` makes a real table. One engine per app, one session per request.
- Credentials come from `.env` via `load_dotenv` + `os.getenv`; build the URL with `URL.create`.
- `exec` returns a box → `.first()` / `.all()`; `session.get(Model, id)` returns the object or None.
- Create = `model_validate` → `add` → `commit` → `refresh`; update with `exclude_unset` + `sqlmodel_update`.
- Pydantic → 422, your `HTTPException` → 404, uncaught database errors → 500.
- One model per job: `HeroCreate` in, `HeroPublic` out - the client never sets `id` or sees `secret_name`.